COMP 018: Database Administration - Activity 5 Progress Report

Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	COMP 018 - Database Administration
Evaluating Group	Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	Campus Canteen Pre-Order and Queue Management System
Target Project Group	Group 8 (bwisIT Solutions): Joshua Borreta, Martin Lawrence Bola, Joaquin Phillip Juatchon, Karl Raphael Marcos, Sancho Pacheco Jr, Euan Joseph Cruz
Target Project Repository	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Cantee n-Pre Order-and-Queue-Management-System
Primary Database	canteen_db (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	canteen_db_test_restore

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection

Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 8.5 megabytes across seven standard tables and seven sequences within the public schema. Table storage analysis showed that queue entries occupy 56 kilobytes, vendors and students occupy 48 kilobytes each, notifications occupy 32 kilobytes, and order items, menu items, and customer orders occupy 24 kilobytes each.

2.2 Connection, Session & Activity Monitoring

Database statistics recorded thirteen active connections with 465 committed transactions and 8 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across operational sessions, yielding a transaction rollback ratio below 1.7 percent and confirming healthy transactional behavior. System catalog queries verified that roles and memberships accurately reflect defined security policies.

2.3 Table Statistics & Bloat Inspection

Table usage statistics showed that customer orders, order items, and menu items exhibited zero to minimal dead tuples. System statistics confirmed that PostgreSQL background autovacuum and autoanalyze processes periodically monitor user tables to maintain page efficiency.

2.4 Log Review & Issue Identification

Review of database logs verified that unauthorized access attempts during Activity 4 security testing were logged by the engine, including denied deletion attempts on customer order history. Network port collisions were managed cleanly by remapping application configuration settings. Engine operational logs recorded normal system operations without fatal errors or disk storage warnings.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Permission testing confirmed that runtime application accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.
Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on customer orders, order items, and menu items scanned underlying relations, removing obsolete row versions and refreshing query planner statistics. Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application transactions and disrupt ordering workflows.

2.6 Query Performance Analysis

Performance analysis was conducted on a vendor order lookup query joining customer orders, student details, vendor stalls, and queue entries. The query executed in 0.147 milliseconds with complete buffer cache hit efficiency. The query planner selected a sequential scan over an index scan because the relation fits within a single memory buffer page, avoiding index page processing overhead. Should transaction volume grow significantly in production, adding a composite index on vendor identifiers and order dates will maintain performance.

2.7 Prioritized Recommendations & Technical Justification

The current index structure is appropriate for active workload demands, as queries execute under 0.15 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary. As transaction volume scales past 10,000 orders, establishing a composite index covering vendor IDs and order dates will optimize query execution. Setting autovacuum scale factors on high-frequency order tables will automate background cleanup, while assigning maintenance roles to dedicated accounts will preserve security boundaries.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 8 - bwisIT Solutions: Campus Canteen Pre-Order and Queue Management System
Database inspected	canteen_db (PostgreSQL 18.6)
Account and role used	canteen_admin (DBA Maintenance), canteen_app (Runtime), canteen_report_user (Audit)
Health check result	Healthy (0 conflicts, 0 deadlocks, 8.5 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on customer_order, order_item, menu_item via canteen_admin
Query analyzed	Vendor incoming orders lookup with student and queue join (Seq Scan + Hash Join, < 0.15 ms)
Recommendation priority	No action required (Follow-up index on customer_order(vendor_id, order_date DESC) when orders exceed 10,000)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(04/11/2026)
PDF progress document	Activity5_Progress_Report_Group8.pdf
Last updated	2026-11-04

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.